ModelBoundary Lab: An Evidence-Gated Protocol for Evaluating Local Language-Model Isolation

Ayoub Issa
Independent Researcher

Version 0.4.0 — 17 October 2026

Abstract

We present ModelBoundary Lab (MBL), a reproducible protocol scaffold for evaluating whether a local language-model process is separated from host resources. MBL separates observed operating-system properties, request-policy conformance, model inference, and confidentiality claims. A C++ probe records process namespace identifiers and selected `/proc` status; TypeScript validates evidence records; a Rust preflight prototype fails closed when its limited preconditions are not met; and an optional runner performs a local inference smoke test. The current build demonstrates tool compilation and limited process observations only. It does not establish a complete sandbox, real-model execution, or neural-weight confidentiality. The paper makes this evidence boundary explicit.

1 Problem and scope

A model-serving boundary must prevent an untrusted or compromised inference workload from reading unauthorized host data, gaining network access, or exposing protected model artifacts. These are distinct properties. Output filtering cannot substitute for kernel isolation, and a namespace identifier alone does not prove that a process lacks access to host resources. MBL is a measurement and reproducibility framework, not a certified sandbox.

2 Threat model

Let M denote model parameters, X authorized inputs, Y observable outputs, and H host resources. An adversary controls prompts and may exploit the serving process, but is not assumed to control host kernel or administrator privileges. The assets include model weights, host files, credentials, and network reachability. The adversary may attempt path traversal, prompt-based extraction, filesystem reads, outbound exfiltration, resource exhaustion, namespace confusion, or policy bypass. Host-root compromise, hardware side channels, and supply-chain compromise are outside the current guarantee.

3 Formalization

Let $\mathcal{A}$ be an adversary strategy and $\mathsf{View}_{\mathcal{A}}$ its observations. Define unauthorized disclosure event D as a view containing a protected parameter fragment beyond the declared output policy. A confidentiality goal is

$$\Pr[D \mid \mathcal{A}, \mathcal{T}] \leq \epsilon,$$

where $\mathcal{T}$ states the tested environment, attack budget, oracle, and model revision. This inequality is a target, not a measured result in this release. Let $C_i \in \{0, 1, \perp\}$ encode each required control as verified, failed, or inconclusive. The release gate is

$$G = \prod_{i=1}^{k} \mathbf{1}[C_i = 1].$$

Any missing or inconclusive mandatory control makes $G = 0$; there is no silent fallback. For empirical attack trials with independent Bernoulli outcomes Z_j, an estimate $\hat{p} = n^{-1} \sum_j Z_j$ may be accompanied by a binomial interval only if the independence and sampling assumptions are justified. Replaying a deterministic fixture does not create independent trials.

4 Architecture and protocol

The pipeline records (1) pinned model and tokenizer identity, (2) host/kernel and toolchain metadata, (3) process and namespace observations, (4) network and filesystem negative controls, (5) raw logs and exit codes, and (6) cryptographic hashes for artifacts. Evidence records use explicit classes: `synthetic`, `observed`, and `model-tested`. The classes are not interchangeable. The optional inference runner requires an immutable revision and records output and accessible weight-file hashes; it does not establish confidentiality.

5 Current implementation and observations

The C++ probe compiles and records namespace links and selected process status. Its observations describe the current process only; comparison to a host observer is required to demonstrate separation. The TypeScript library validates evidence schemas and rejects a confidence interval request when independence is not documented. The Rust component is intentionally a preflight prototype, not a complete sandbox: it does not itself create namespaces, install a seccomp filter, or configure Landlock. It refuses to launch absent explicit preconditions, and its attestation variable is not a security proof.

Component	Evidence status in this package
C++ probe	Built; records current-process observations
TypeScript tools	Source included; build requires npm dependency installation
Rust preflight	Source included; Rust toolchain absent in build runtime, uncompiled here
Local pretrained inference	Not run; requires external model weights and dependencies
Landlock enforcement	Not established; do not infer from namespace tests
Weight confidentiality	Not tested

Table 1: Implementation status; absence of evidence is not a pass.

6 Limitations

This release does not prove host filesystem separation, seccomp enforcement, Landlock confinement, capability removal, egress denial under all protocols, resistance to privileged host actors, or absence of side channels. A smoke test confirms only that a model can answer a prompt in the measured environment. The custom research license applies only to project-owned material and cannot override third-party model licenses.

7 Reproducibility

Commands and evidence schema are in the repository. Each run should preserve raw output, immutable model revision, hashes, kernel version, exact command, and exit status. Failed, skipped, and inconclusive checks must remain distinct. Independent replication on another host is required before generalizing beyond the measured environment.

8 Conclusion

MBL provides an evidence-gated structure for a more rigorous evaluation, not a claim that the evaluation is complete. The next milestone is to run a pinned local model under a separately reviewed, kernel-enforced sandbox, compare host and guest observations, and publish raw reproducibility artifacts. Only those experiments can support a bounded confidentiality claim.